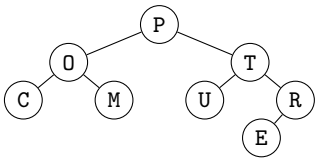
CSCI-UA 102 — Quiz 5

Binary Trees

Name: _____ NetID: _____

I encode a word in a binary tree as follows: for "COMPUTER", I first pick a random letter, say "P", and put `P` at the root, `COM` on the left, and `UTER` on the right. Then I repeat at each child. Continuing, I get:



You may use these interfaces:

```
public interface Position<E> {
    E getElement();
}
public interface BinaryTree<E> {
    Position<E> root();
    Position<E> left(Position<E> p);
    Position<E> right(Position<E> p);
    boolean isEmpty();
}
```

Write an efficient, recursive method that recovers the original word from the tree:

```
static String wordFromTree(BinaryTree<Character> tree)
```

Time complexity: _____

Solution

Key insight: The encoding places all letters that come *before* a node's letter in its left subtree, and all letters that come *after* in its right subtree. This is exactly an **in-order traversal**: left subtree, then root, then right subtree.

```
static String wordFromTree(BinaryTree<Character> tree) {
    if (tree.isEmpty()) return "";
    return inOrderWord(tree, tree.root());
}

static String inOrderWord(BinaryTree<Character> tree,
                          Position<Character> p) {
    if (p == null) return "";
    String leftPart = inOrderWord(tree, tree.left(p));
    String rightPart = inOrderWord(tree, tree.right(p));
    return leftPart + p.getElement() + rightPart;
}
```

Time complexity: $O(n^2)$ due to string concatenation at each node ($O(n)$ per concat, n nodes). $O(n)$ if using `StringBuilder`.